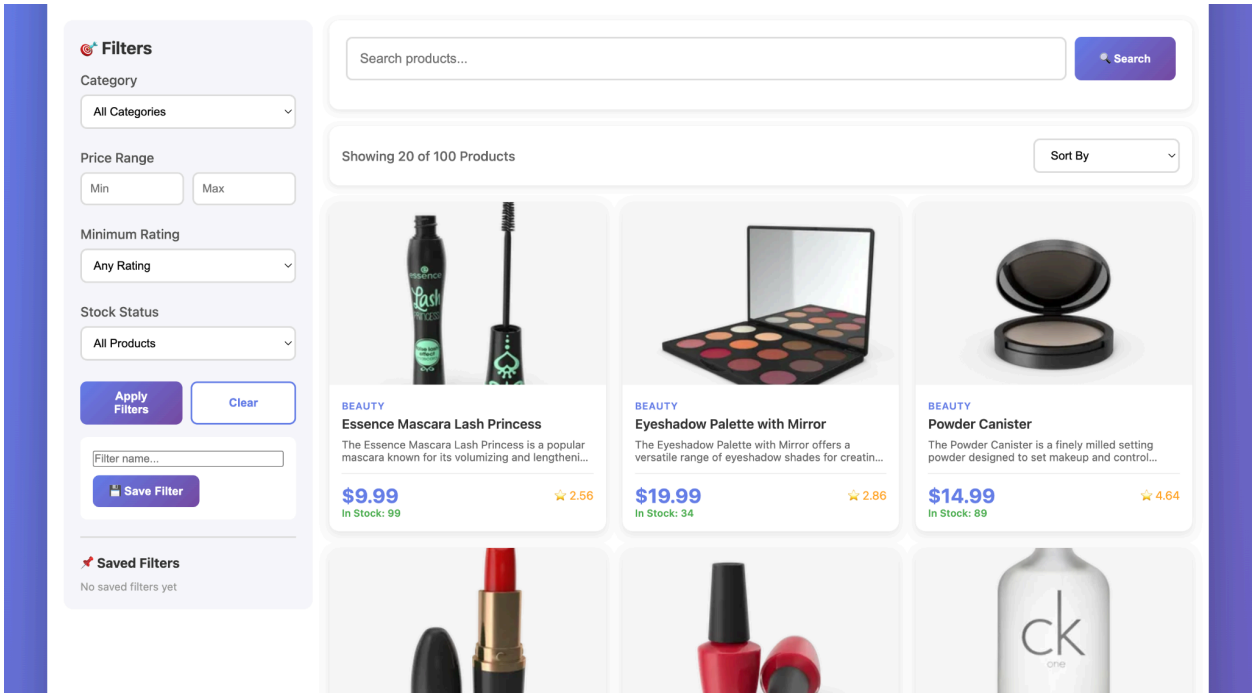
Coding Assignment: Product Catalog with Local-First State



Objective

Your task is to build a highly responsive, performant **Product Catalog Page** based on the provided UI design. The application must handle data locally from a provided JSON file, emphasizing efficient state management, filtering logic, and clean architectural patterns.

Core Requirements

1. Data Integration

- **Source:** Use a local JSON file containing a list of products.
- **Loading:** Implement a data fetching layer that simulates an asynchronous load, even though the data is local.

2. Search & Filter System (Local Implementation)

All filtering and searching must happen on the client-side without page reloads.

- **Global Search:** A text input to filter products by name or description in real-time.
- **Category Filter:** A dropdown to narrow products by their specific category.
- **Price Range:** Two numeric inputs (Min/Max) to filter products within a price bracket.
- **Minimum Rating:** A dropdown to show products with a rating equal to or higher than the selection.
- **Stock Status:** Toggle between "All Products", "In Stock", and "Out of Stock".

- **Clear All:** A button to reset all active filters to their default state.

3. State Persistence (Saved Filters)

- **Save Filter:** Users should be able to name their current combination of filters and save them.
- **Persistence:** Saved filters persist across browser refreshes.
- **List View:** Display saved filters in the sidebar; clicking one should immediately apply those settings to the product grid.

4. Product Display & Sorting

- **Product Grid:** Display product cards featuring the image, category label, title, short description, price, stock count, and star rating.
- **Pagination/Counter:** Show the count of currently visible products versus the total (e.g., "Showing 20 of 100 Products").
- **Sorting:** Implement a "Sort By" dropdown for:
    - Price: Low to High / High to Low.
    - Rating: High to Low.

Evaluation Criteria

Category	High Priority Focus
Feature Completeness	Does every UI element function as expected?
Code Organization	Use of modular components, custom hooks, and separation of concerns.
Performance	Efficient rendering, performant image loading.
Clean Code	Meaningful variable naming, consistent formatting, and self-documenting code.

Bonus Points

- **Accessibility (a11y):** Proper use of ARIA labels, keyboard navigation for filters etc.
- **UX/UI Polishing:** Implementation of skeleton loaders, empty states (when no products match filters), and smooth transitions.
- **Scalability:** How easily could this system be swapped from a JSON file to a real REST/GraphQL API?
- **Testing:** Inclusion of unit tests for the filtering/sorting utility functions.

Submission Guidelines

- Git repository with a README with setup instructions.
- Deployed on a cloud service.
- Candidates are free to use frameworks and libraries of their choice.
- **AI/LLM Usage Policy:** Candidates are encouraged to use AI coding assistants or Large Language Models (LLMs) (e.g., GitHub Copilot, Gemini, ChatGPT) for tasks such as boilerplate generation, utility function creation, or bug fixing. **However, the candidate must fully own, understand, and be able to explain every line of code submitted.** Simply submitting unreviewed AI-generated code that the candidate cannot articulate will be considered a failure of the assignment.